Lab 7: Fuzzing for Input Validation Bugs

Name: Guru R Bharadwaj
SRN: PES1UG24CS177
Section: 5 C
Assigned repository: SETAPESU26/L6_L7_48_daycare_fees

Objective

To explore fuzz testing as a security and quality assurance technique by finding and fixing bugs in the given Python script with the Hypothesis fuzzing library.

The program

The script for my repository is `daycare_fees.py`, which plays the part of `processor.py` from the handout. It has three functions. `parse_enrolment` takes text like `toddler:25` and returns the age group and hours. `hours_average` takes weekly hours as comma separated values and returns the average. `payable_fee` takes `fee|discount`, adds 18% GST and subtracts the discount. The test file is `test_daycare_fees.py`, the equivalent of `test_processor.py`.

Step 1: Installing dependencies

I installed both libraries with `pip install hypothesis` and `pip install pytest`. The versions used were Python 3.13, pytest 9.1.1 and Hypothesis 6.169.1.

Step 2: Running the buggy program

I ran the script three times. With the sample inputs from the prompts it works. Typing `toddler` without the colon crashes it with a ValueError, and so does a blank value in the hours list such as `20,,30`. The first run also gives an average of 25 for 20, 25 and 30, which happens to be right, but only because the code uses floor division.

```
Windows PowerShell

PS C:\Users\Temp\Desktop\SE-Experiential-Learning\Lab-7\daycare_fees> python daycare_fees.py
==== Running parse_enrolment ====
Enter age group:hours (e.g. toddler:25): toddler:25
Age Group and Hours: ('Toddler', 25.0)

==== Running hours_average ====
Enter weekly hours as CSV (e.g. 20,25,30): 20,25,30
Average Hours: 25

==== Running payable_fee ====
Enter fee|discount (e.g. 3000|250): 3000|250
Payable Fee: 3290.0

PS C:\Users\Temp\Desktop\SE-Experiential-Learning\Lab-7\daycare_fees> python daycare_fees.py
==== Running parse_enrolment ====
Enter age group:hours (e.g. toddler:25): toddler
Traceback (most recent call last):
  File "C:\Users\Temp\Desktop\SE-Experiential-Learning\Lab-7\daycare_fees\daycare_fees.py", line 31, in <module>
    main()
    ~~~~^^
  File "C:\Users\Temp\Desktop\SE-Experiential-Learning\Lab-7\daycare_fees\daycare_fees.py", line 17, in main
    result = parse_enrolment(user_input)
  File "C:\Users\Temp\Desktop\SE-Experiential-Learning\Lab-7\daycare_fees\daycare_fees.py", line 2, in parse_enrolment
    age_group, hours = text.split(':')
    ^^^^^^^^^^^^^^^^
ValueError: not enough values to unpack (expected 2, got 1)

PS C:\Users\Temp\Desktop\SE-Experiential-Learning\Lab-7\daycare_fees> python daycare_fees.py
==== Running parse_enrolment ====
Enter age group:hours (e.g. toddler:25): infant:20
Age Group and Hours: ('Infant', 20.0)

==== Running hours_average ====
Enter weekly hours as CSV (e.g. 20,25,30): 20,,30
Traceback (most recent call last):
  File "C:\Users\Temp\Desktop\SE-Experiential-Learning\Lab-7\daycare_fees\daycare_fees.py", line 31, in <module>
    main()
    ~~~~^^
  File "C:\Users\Temp\Desktop\SE-Experiential-Learning\Lab-7\daycare_fees\daycare_fees.py", line 22, in main
    result = hours_average(user_input)
  File "C:\Users\Temp\Desktop\SE-Experiential-Learning\Lab-7\daycare_fees\daycare_fees.py", line 6, in hours_average
    values = [int(r) for r in hours_csv.split(',')]
              ~~~^^^
ValueError: invalid literal for int() with base 10: ''
```

Figure 1: Output of python daycare_fees.py before the fix

Step 3: Writing the Hypothesis tests

The starter file only had the three `@given` decorators with empty function bodies. I filled in a no-crash test for each function. Each one feeds in random text or None, fails if anything raises, and checks that a returned value is sensible: hours and fees must be finite and not negative, and the age group cannot be empty.

Not crashing does not prove the answer is right, so I added three more tests built from valid input. One makes lists of hours from 0 to 168 and compares `hours_average` with the real mean. One checks that a known age group and hour count come back unchanged. The last compares `payable_fee` with the expected amount rounded to two decimals.

Step 4: Running the tests on the buggy code

Five of the six tests failed. Only the valid enrolment test passed.

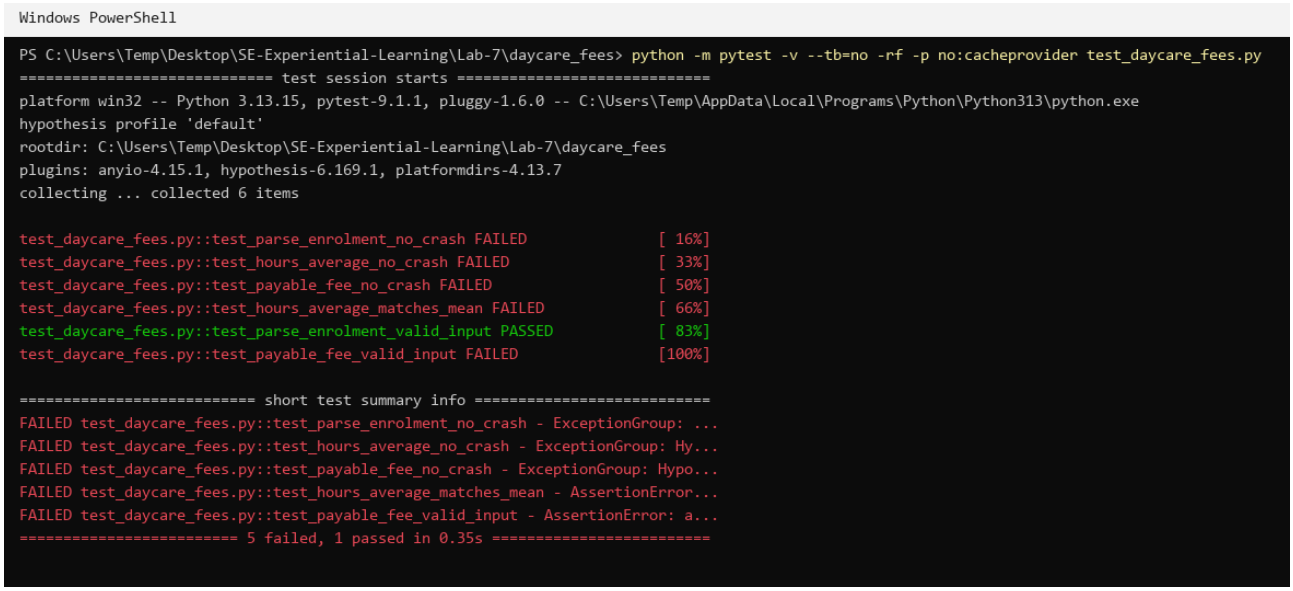


Figure 2: python -m pytest on the original code

Filtering the output down to the assertion messages shows the smallest inputs Hypothesis found for each failure:

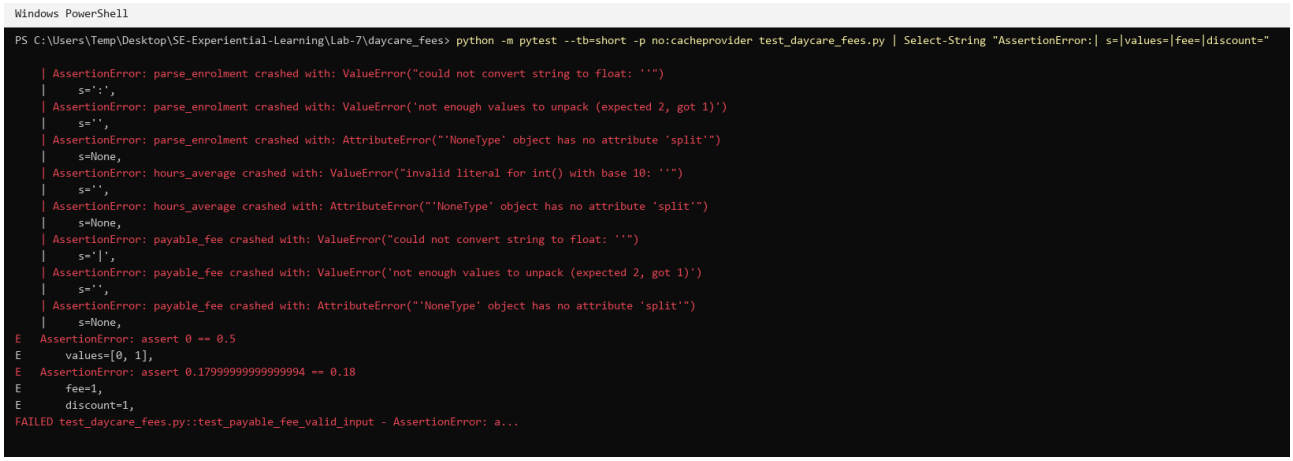


Figure 3: Minimal failing inputs reported by Hypothesis

Function	Failing input	What went wrong
parse_enrolment	None	AttributeError, None has no split
parse_enrolment	''	No colon, so the unpack fails
parse_enrolment	':'	Empty hours, float('') fails
hours_average	None	AttributeError
hours_average	''	int('') fails on the empty item
hours_average	[0, 1]	Returns 0 instead of 0.5 because of //
payable_fee	None	AttributeError
payable_fee	''	No pipe, so the unpack fails

payable_fee	'|'	Empty fee, float('') fails
payable_fee	1|1	Returns 0.17999999999999994, not 0.18

A few other problems never showed up as a shrunk example but were still in the code. `float` accepts strings like `nan`, `inf` and `-5`, so the old code would happily return negative or infinite hours, or a fee of nan, since `max(nan, 0)` gives nan. A very large fee such as `1e309` becomes infinity. An age group made of spaces was also accepted.

Step 5: Fixing the program

All three functions now return None for bad input instead of raising, and `main` prints a short message when that happens. The changes were:

1. Each function checks that it got a string before calling `split`.
2. `parse_enrolment` and `payable_fee` need exactly one separator, so `a:b:c` and `1|2|3` are rejected too.
3. A helper, `_to_number`, strips the text, rejects empty strings, catches the ValueError from `float`, and rejects nan, infinity and negative numbers.
4. Hours have to be between 0 and 168, the number of hours in a week, and the age group cannot be blank.
5. `hours_average` uses true division and rounds to two decimals, so 0 and 1 give 0.5. It also accepts decimal hours like 22.5.
6. `payable_fee` rounds the result to two decimals and returns None if the total overflows to infinity.

Running the fixed script with the same inputs that used to crash now gives a message for each bad value, and the correct answers for 0,1 and 1|1.

```
Windows PowerShell

PS C:\Users\Temp\Desktop\SE-Experiential-Learning\Lab-7\daycare_fees> python daycare_fees.py
==== Running parse_enrolment ====
Enter age group:hours (e.g. toddler:25): toddler:25
Age Group and Hours: ('Toddler', 25.0)

==== Running hours_average ====
Enter weekly hours as CSV (e.g. 20,25,30): 20,25,30
Average Hours: 25.0

==== Running payable_fee ====
Enter fee|discount (e.g. 3000|250): 3000|250
Payable Fee: 3290.0

PS C:\Users\Temp\Desktop\SE-Experiential-Learning\Lab-7\daycare_fees> python daycare_fees.py
==== Running parse_enrolment ====
Enter age group:hours (e.g. toddler:25): toddler
Invalid input, expected something like toddler:25

==== Running hours_average ====
Enter weekly hours as CSV (e.g. 20,25,30): 20,,30
Invalid input, expected numbers between 0 and 168 separated by commas

==== Running payable_fee ====
Enter fee|discount (e.g. 3000|250): 3000
Invalid input, expected two non-negative numbers like 3000|250

PS C:\Users\Temp\Desktop\SE-Experiential-Learning\Lab-7\daycare_fees> python daycare_fees.py
==== Running parse_enrolment ====
Enter age group:hours (e.g. toddler:25): infant:-5
Invalid input, expected something like toddler:25

==== Running hours_average ====
Enter weekly hours as CSV (e.g. 20,25,30): 0,1
Average Hours: 0.5

==== Running payable_fee ====
Enter fee|discount (e.g. 3000|250): 1|1
Payable Fee: 0.18

PS C:\Users\Temp\Desktop\SE-Experiential-Learning\Lab-7\daycare_fees> python daycare_fees.py
==== Running parse_enrolment ====
Enter age group:hours (e.g. toddler:25): :nan
Invalid input, expected something like toddler:25

==== Running hours_average ====
Enter weekly hours as CSV (e.g. 20,25,30): 20,inf
Invalid input, expected numbers between 0 and 168 separated by commas

==== Running payable_fee ====
Enter fee|discount (e.g. 3000|250): 1e309|0
Invalid input, expected two non-negative numbers like 3000|250
```

Figure 4: Output of python daycare_fees.py after the fix

Step 6: Re-running the tests

All six tests pass against the fixed code. I ran the suite a few more times, since Hypothesis picks new inputs each run, and it stayed green.

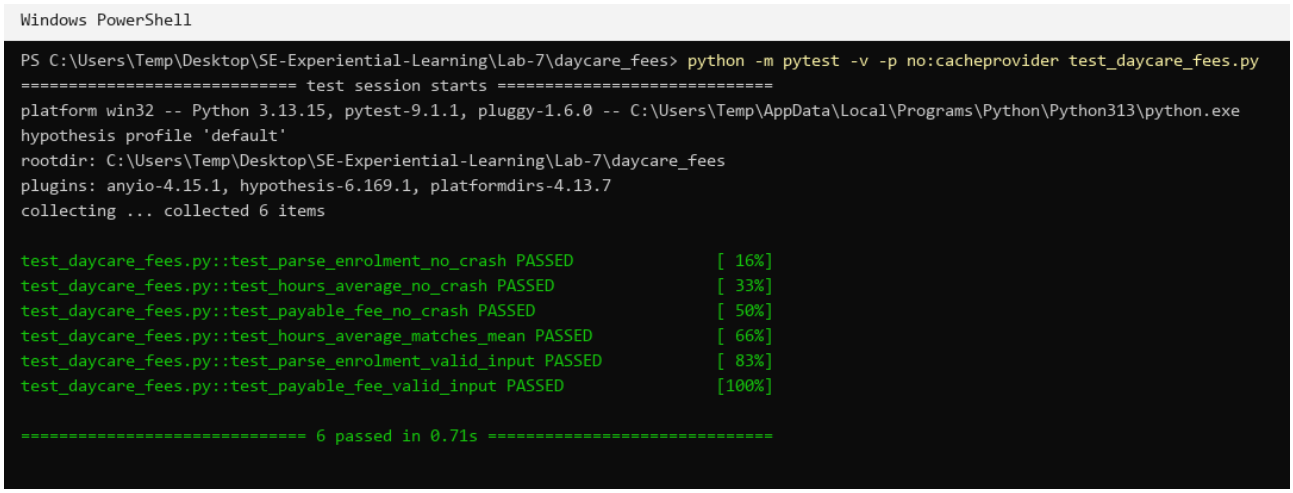


Figure 5: python -m pytest on the fixed code

Conclusion

The original script crashed or gave a wrong answer on ten different inputs, and the six tests found every one in under a second. My own test cases would have caught the missing colon. I doubt they would have caught the floor division or the 0.17999 rounding error, because the sample input 20,25,30 averages to a whole number and hides both. Most of the fix was rejecting bad input before it reached `float` or `int`. The rest was getting the arithmetic right.